

3. Bubble Sort with Big-O Analysis

Aim: To sort an array using Bubble Sort and analyze its time complexity using Big-O notation.

Algorithm

1. Read the array.
2. Compare adjacent elements.
3. If the first element is greater than the second, swap them.
4. Continue until the largest element reaches the end.
5. Repeat for all remaining elements.
6. Print the sorted array.

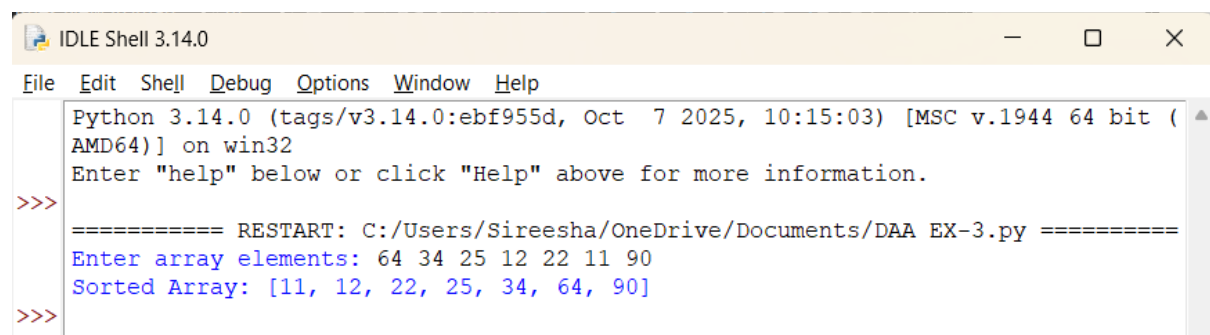
Python Program

```
arr = list(map(int, input("Enter array elements: ").split()))
n = len(arr)
for i in range(n):
    for j in range(0, n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
print("Sorted Array:", arr)
```

Input

Enter array elements: 64 34 25 12 22 11 90

Output



```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Sireesha/OneDrive/Documents/DAA EX-3.py =====
Enter array elements: 64 34 25 12 22 11 90
Sorted Array: [11, 12, 22, 25, 34, 64, 90]
>>>
```

Result

Thus, the Python program to sort the array using Bubble Sort was successfully executed, and its Big-O time complexity was analyzed.